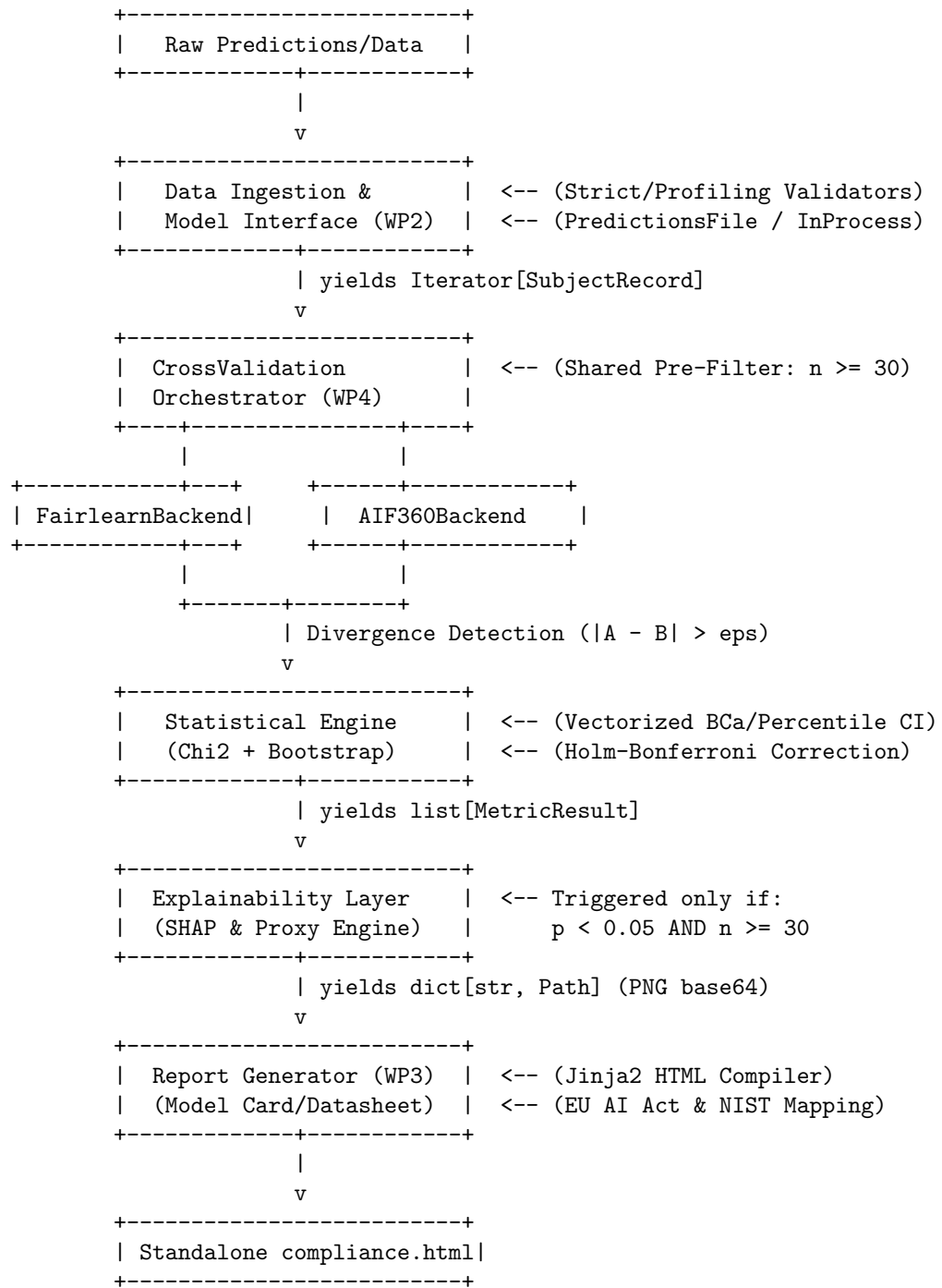


BiasAperture — Research Sprint: Mid-Level Architectural Design

Target Audience:	Lead Software Engineers, Machine Learning Engineers, Code Reviewers
Document Level:	Mid-Level (System Architecture, Component Interactions, Strategy Patterns & Contracts)
Date:	August 2026
Context:	Milestone M1 Completion & 20-Track Parallel Research Sprint

1 System Topology & Component Overview

BiasAperture implements a decoupled, pipeline-oriented architecture designed around explicit data contracts and the Strategy pattern. Each subsystem encapsulates specific responsibilities while communicating across locked data schemas.



2 Component Design & Interfacing

2.1 Module 1: Ingestion & Invariant Validation (`data_ingestion.py`, `model_interface.py`)

The ingestion module transforms heterogeneous prediction outputs into standardized `SubjectRecord` streams.

Ingestion Contracts & Patterns

- `ModelInterface (ABC)`: Defines `get_predictions() -> Iterator[SubjectRecord]`.

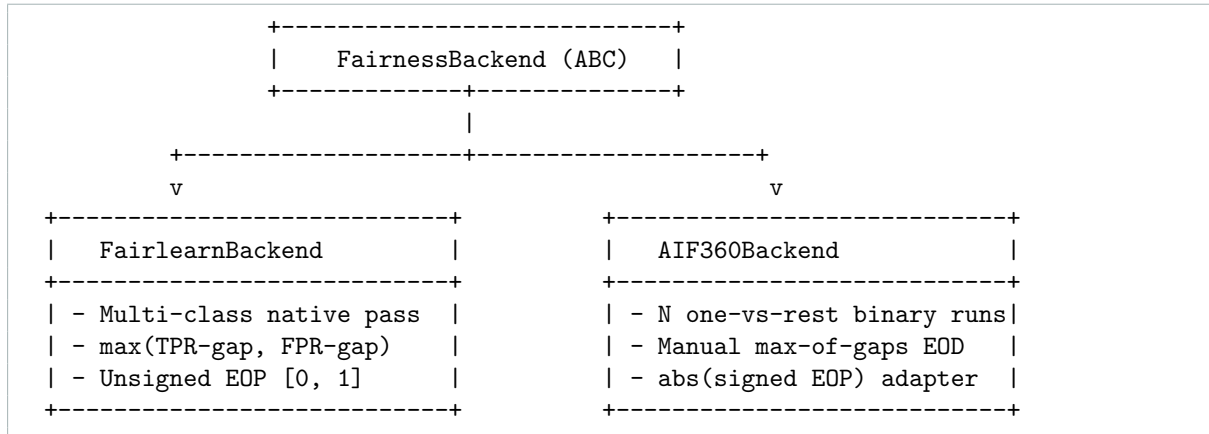
- **PredictionsFileInterface:** Ingests CSV or JSON predictions files. Maps FairFace output columns (`face_name_align`, `race`, `gender`, `age`) and caller-specified task columns (`true_label`, `predicted_label`).
- **InProcessInterface:** Direct PyTorch evaluation hook executing tensor preprocessing (dlib CNN 5-point crop $\rightarrow 300 \times 300 \rightarrow$ resize $224 \times 224 \rightarrow$ ImageNet normalize).

Two-Mode Validation Engine

1. **Strict / Fail-Fast Mode:** Used in production audit runs. Immediately raises `SchemaError` or `ValueError` upon encountering missing mandatory columns, unmapped demographic labels, corrupt `image_ids`, or contradictory duplicates.
2. **Permissive / Profiling Mode:** Used during exploratory data ingestion. Collects all row-level anomalies, NaN distributions, and syntax drifts into a structured `ValidationSummary` diagnostic report without halting execution.

2.2 Module 2: Fairness Strategy Engine (`bias_aperture/fairness/`)

To guarantee methodological independence, metric evaluation is structured via the **Strategy Pattern**.



Shared Base Architecture (`fairness/base.py`)

- **Single Source of Sample Sizes:** `subgroup_sample_sizes()` and `is_insufficient()` are defined once in `base.py`. Both backends share this calculation to prevent false divergences caused by divergent internal pandas/numpy grouping logic.
- **CrossValidationOrchestrator:** Accepts a sequence of backends (`Sequence[FairnessBackend]`). Evaluates metrics across all backends and flags divergence when $|v_1 - v_2| > \epsilon_{\text{metric}}$.
- **Configurable Tolerances (ϵ):**
 - Difference metrics (DPD, EOD, EOP): $\epsilon = 0.05$.
 - Ratio metrics (DIR): $\epsilon = 0.10$.

2.3 Module 3: Statistical Rigor Engine (`fairness/statistics.py`)

Every metric computation is validated by statistical confidence and hypothesis testing.

1. Bootstrap Uncertainty Engine:

- Resamples each demographic subgroup independently with replacement ($B \geq 1,000$).

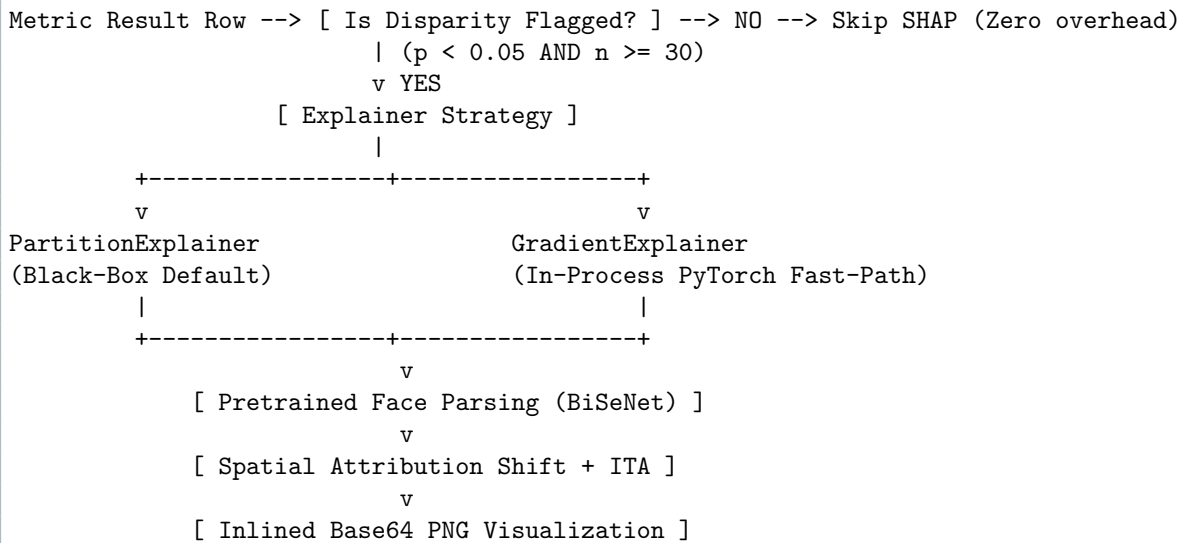
- Vectorized index generation via `numpy.random.Generator` seeded with distinct `SeedSequence` keys per `(metric_name, subgroup)`.
- Uses **BCa (Bias-Corrected and Accelerated)** percentiles with automatic fallback to standard percentile bootstrap when acceleration terms degenerate near boundaries ($n \approx 30$).

2. Chi-Squared Independence Testing:

- Constructs $2 \times K$ contingency tables from observed prediction outcomes.
- Evaluates independence using `scipy.stats.chi2_contingency` with Yates' continuity correction.
- Applies **Holm-Bonferroni step-down FWER adjustment** across subgroup hypothesis families.
- Automatically falls back to Fisher's Exact Test for 2×2 tables with expected cell counts < 5 .

2.4 Module 4: Targeted Explainability & Proxy Detection (`explainability.py`)

Explainability is executed **conditionally** as a targeted post-processing stage to minimize compute overhead.



- **Selective Triggering:** Only subgroups failing the fairness audit ($p < \alpha$, $n \geq 30$) trigger SHAP. Sample size is capped at $k = \min(n, 20)$ representative exemplars.
- **Explainer Selection:**
 - **PartitionExplainer:** Default model-agnostic explainer. Operates on pure input/output probability vectors without requiring model gradients.
 - **GradientExplainer:** High-speed GPU-accelerated path used when direct PyTorch tensor graphs are supplied via `InProcessInterface`.
- **Proxy Detection:**
 - Segment facial regions into semantic masks (skin, hair, eyes, nose, background) using an off-the-shelf face parser.

- Compute regional SHAP mass and calculate **Individual Typology Angle (ITA)** colorimetry on skin segments.
- Quantify if the model shifts attribution mass to demographically-correlated features (e.g. skin tone) during classification.

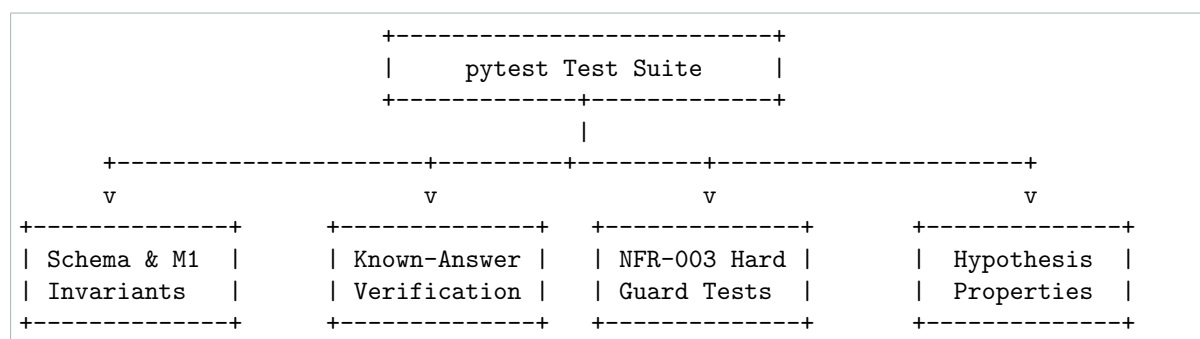
2.5 Module 5: Compliance Report Generation (report/)

The reporting engine translates technical metrics and governance metadata into a self-contained compliance dossier.

- **Standalone HTML Design:** Outputs a single `.html` document with embedded CSS, base64-encoded SHAP plots, and zero external CDN/JS dependencies.
- **Mitchell et al. (2019) Model Card:** Auto-populates all 9 core sections:
 1. *Model Details*: Metadata, version, architecture.
 2. *Intended Use*: Scoped applications and out-of-scope boundaries.
 3. *Factors*: Locked demographic axes (Race, Gender, Age).
 4. *Metrics*: Core Four metrics, significance thresholds, confidence levels.
 5. *Evaluation Data*: FairFace benchmark details.
 6. *Training Data*: Explicit “N/A — Diagnostic Audit Tool” boundary statement.
 7. *Quantitative Analyses*: Unitary and intersectional metric tables.
 8. *Ethical Considerations*: Proxy attribution caveats, representation analysis.
 9. *Caveats & Recommendations*: Limitations and guidance for deployers.
- **Gebru et al. (2018) Dataset Datasheet:** Documents FairFace provenance, Flickr CC licensing, AMT annotation consensus, and perceived demographic categorization.
- **Regulatory Tracing:** Embeds static lookup tables linking each metric to EU AI Act Article 10(2)–10(5) and NIST AI RMF Measure subcategories.

3 Comprehensive Testing & Verification Architecture

The test suite in `src/tests/` verifies correctness, safety invariants, and mathematical precision.



1. Schema Invariant Unit Tests (test_schema.py):

- Verifies dataclass freezing, slot enforcement, and label literal membership.
- Enforces that any attempt to instantiate `MetricResult(subgroup_sample_size=25, metric_value=0.12)` immediately raises `ValueError`.

2. Known-Answer Deterministic Tests:

- An 8-record ground-truth synthetic block with hand-calculated expected values:

$$\text{DPD} = 0.500, \quad \text{EOD} = 0.500, \quad \text{EOP} = 0.500, \quad \text{DIR} = 0.333$$

- Replicated $\times 8$ ($n = 64$) to test the end-to-end statistically gated pipeline.

3. NFR-003 Engine Guard Verification:

- Uses `unittest.mock.patch.object` on statistical routines to prove that backend algorithms **never execute computations** for $n < 30$ groups, preventing wasted CPU cycles.

4. Hypothesis Property-Based Tests:

- Generates arbitrary synthetic `SubjectRecord` populations to prove:
 - All difference metrics remain strictly bounded in $[0, 1]$.
 - No `NaN` or `Inf` floats ever escape into `MetricResult.metric_value`.
 - Confidence interval widths shrink monotonically as n increases.